

Test Report — `apply_schema.py`, the deploy step that applies schema changes

PR: kuantorflow#181 + kuantorflow_automation#39 · Issue: kuantorflow#180 · Date: 2026-08-02

Summary

Deploys could not add a column: `schema.sql` uses `CREATE TABLE IF NOT EXISTS`, and the `ALTER` statements lived in it as comments, so a deploy that followed the instructions faithfully still skipped them — which took card saving down in production earlier the same day.

`apply_schema.py` now applies both halves in one idempotent, self-reporting command. **370 offline tests pass** (was 353), plus 5 opt-in round-trip tests against a real MySQL.

What changed

- ``kuantorflow/apply_schema.py`` (new) — reads `schema.sql` for tables and holds an ordered MIGRATIONS list for changes to tables that already exist (`pos`, `added_by_user_id`, `idx_added_by`, `fk_flashcards_user`). Each step names the object it creates; existence of that object is the idempotency check, so no `schema_migrations` table is needed at this size. Reports every step (+ applied / = already present / ~ would be applied), exits non-zero on failure naming the step and the statement, and takes `--dry-run`.
- ``kuantorflow/schema.sql`` — `CREATE TABLE` only; the four commented-out `ALTER`s are gone, replaced by a pointer to the script. A bare `ALTER` left in the file is now a loud `SchemaError` rather than a comment nobody runs.
- ``kuantorflow/README.md``, ``kuantorflow/CLAUDE.md`` — the deploy step is `python apply_schema.py`, not `mysql < schema.sql`; adding a column is documented as two edits (`schema.sql` for new databases, a migration for the existing one).

Automated tests

- **Added:** `tests/test_apply_schema.py` — 17 offline tests against a fake database that derives what a statement created **from the statement itself**, so a migration whose SQL does not build the object its target names surfaces as a step that keeps re-applying. Covers: a commented-out `ALTER` is not a statement (the bug itself); a bare `ALTER` in `schema.sql` raises and points at MIGRATIONS; a fresh database needs no migrations on top of its `CREATE TABLE`s; a pre-#89 database gains column → index → key in that order; a half-applied database finishes the rest; a second run executes nothing; `--dry-run` touches nothing; a failure names its step; main returns 0/1 and closes the connection either way.
- **Added:** `tests/test_apply_schema_db.py` — 5 db-marked tests running the real script against a real local MySQL, opt-in via `RUN_DB_ROUNDTRIP=1`. They create and drop their **own** scratch database (`kuantorflow_apply_schema_test`) and never touch the configured one.
- **Suite result:** `pytest -m "not live"` — **370 passed, 6 skipped** (was 353 passed, 1 skipped); the 5 new skips are the opt-in db tests. With `RUN_DB_ROUNDTRIP=1`: `pytest -m db` — **5**

passed.

Manual & browser verification

No UI surface — this is a deploy step, so verification is CLI, against a throwaway local database (kuntorflow_180_probe, dropped afterwards; the real local database was only ever read).

- Empty database, first run → + anonymous_usage, + users, + flashcards, all four migrations already present, 3 change(s) applied, 4 already in place, exit 0. The two halves of the schema agree.
- Same database, second run → nothing to do - 7 object(s) already in place, exit 0.
- flashcards replaced with its pre-#89 shape (no pos, no owner column) with one card in it, --dry-run → 4 change(s) pending, and SHOW COLUMNS confirmed the database was **unchanged**.
- Then applied for real → all four + applied, exit 0. SHOW COLUMNS gives pos directly after word and added_by_user_id directly after topic, matching a table built fresh from schema.sql; idx_added_by and fk_flashcards_user present; the existing card still there as (1, 'brittle', None, None).
- Re-run → nothing to do - 7 object(s) already in place, exit 0.
- Failure path: added_by_user_id planted as VARCHAR(10) so the foreign key cannot be created → exit 1, stderr naming flashcards.fk_flashcards_user failed: 3780 (HY000) ... with the statement and Nothing after this step was applied - fix it and re-run. The steps before it are reported as applied, so the log says exactly how far it got.
- Real local development database, --dry-run → nothing to do - 7 object(s) already in place, which is what the first PythonAnywhere run should print too, since #164's table and #89's column are already in place there by hand.
- Cleanup: probe database dropped, and SHOW DATABASES confirmed no scratch database was left behind after the suite ran.

Result

Pass. Every acceptance criterion in #180 is met: one command applies tables and column changes, a second run reports nothing to do, each change and skip is reported, failure exits non-zero, the four existing migrations are covered, and the README and CLAUDE.md point at the script.

Follow-ups, deliberately not in this PR:

- **The deploy itself.** The script has not yet been run on PythonAnywhere; the first run there is expected to be a no-op and is itself the last check that the idempotency holds against the production database.
- **#126** (blocked_at) is the first migration that will be written under this scheme rather than retrofitted into it.
- A schema_migrations tracking table stays deferred until the list is long enough that re-checking it is slow.